April 13, 2026

CE4011 Assignment 3 Report

Special Topics in Civil Engineering:
Structural Analysis Software Development

GitHub: `https://github.com/ImranMETU/CE4011_Assignment3`

Submitted By

Imran Shahriar

2735835

MSc. in Structural Engineering

2nd Semester

Submitted To

Prof Dr Ozgur Kurc

Contents

1 Object-Oriented Structural Analysis Program **4**
1.1 Introduction 4
1.2 Analysis and Design Decisions 4
1.3 UML Class Diagram 5
1.4 Analysis Assumptions 9
1.5 Input Format 10
1.6 Program Workflow 11
1.7 Output Format 12
1.8 GitHub README Summary 13

2 Testing and Verification **15**
2.1 Testing and Verification Framework 15
2.2 Unit Tests 16
2.3 Interface Tests 16
2.4 Regression Tests 17
2.5 Detailed Test Derivations 17
2.6 Further Tests 21
2.7 Summary of Verification Results 22

3 Warning and Error Handling **24**
3.1 Robustness Strategy 24
3.2 Structure Sample (a) 25
3.3 Structure Sample (b) 28
3.4 Structure Sample (c) 31
3.5 Structure Sample (d) 33
3.6 Structure Sample (e) 35
3.7 Discussion 37

4 Conclusion **38**

5 References **39**

List of Figures

1.1 UML class diagram of the structural analysis program 6

2.1 FTool frame regression screenshot 19
2.2 FTool truss regression screenshot 20
2.3 FTool mixed frame-truss regression screenshot 21

3.1 Structure Sample (a) model geometry 25
3.2 Structure Sample (a) diagnostic output 26
3.3 Verification of Error in FTool for Point/UDL on Structure Sample (a) . . 27
3.4 Structure Sample (b) model geometry 28
3.5 Structure Sample (b) diagnostic output 29
3.6 Verification of Error in FTool for Point/UDL on Structure Sample (b) . . 30
3.7 Structure Sample (c) model geometry 31
3.8 Structure Sample (c) diagnostic output 32
3.9 Structure Sample (d) model geometry 33
3.10 Structure Sample (d) diagnostic output 34
3.11 Structure Sample (e) model geometry 35
3.12 Structure Sample (e) diagnostic output 36

List of Tables

1.1 Summary of class relationship multiplicities 10

2.1 Regression Test 1: frame response comparison 19
2.2 Regression Test 2: truss response comparison 20
2.3 Regression Test 3: mixed structure response comparison 21
2.4 Summary of verification results 22

3.1 Structure Sample (c): FTool and solver comparison 32
3.2 Structure Sample (d): FTool and solver comparison 34
3.3 Structure Sample (e): FTool and solver comparison 35
3.4 Q3 analyzability summary for sample structures 37

1. Object-Oriented Structural Analysis Program

1.1 Introduction

This project develops a 2D structural analysis program using the Direct Stiffness Method (DSM) with an object-oriented design. The implementation supports frame and truss elements in a unified 3-DOF-per-node formulation (u_x, u_y, r_z) and computes nodal displacements, support reactions, and member-end forces under static loading.

The main objective is to provide a reusable and extensible solver architecture for CE4011 assignments, while preserving numerical efficiency by reusing the matrix library from Q1.

File Architecture

To keep the GitHub repository focused and easy to navigate, the project is organized around a small set of source, analysis, and report files:

- **Report root**: `main.tex` assembles the full report, while `Chapter1_...tex` through `Chapter5_...tex` contain the chapter content.
- **Structural solver**: `q2_frame_analysis/` contains the solver implementation, models, tests, regression cases, and generated results.
- **Shared numerics**: `q1_matrix_library/` stores the reusable sparse matrix and solver utilities imported from the earlier assignment.
- **Assets and outputs**: figures, screenshots, and exported logs are kept in dedicated folders such as `images/`, `ftool/`, and `results/` so the codebase stays compact.

This layout reduces clutter, keeps the solver code separated from report assets, and makes the GitHub repository easier to review and maintain.

1.2 Analysis and Design Decisions

Why Object-Oriented Programming (OOP)

OOP was selected to separate responsibilities and improve maintainability:

- **Modeling clarity:** classes such as `Node`, `Element`, and `Structure` map directly to engineering entities.
- **Extensibility:** new element types or load models can be added by subclassing without redesigning the full solver.
- **Encapsulation:** geometry, stiffness, load conversion, and post-processing logic stay close to the associated class.

Why Direct Stiffness Method (DSM)

DSM is the standard finite-element formulation for linear structural analysis:

- naturally supports mixed element assemblies,
- gives a systematic global matrix equation $\mathbf{Kd} = \mathbf{F}$,
- aligns with sparse matrix assembly and iterative solution techniques.

Why Reuse the Q1 Matrix Library

The Q1 implementation (symmetric sparse matrix, vector, and conjugate gradient solver) from Assignment 2 was reused because:

- stiffness matrices are symmetric by formulation,
- sparse storage reduces memory usage,
- conjugate gradient is appropriate for large symmetric systems,
- it preserves continuity between Q1 and Q2 deliverables.

Why a Frame/Truss Hierarchy

A base `Element` abstraction defines shared behavior (geometry transforms, global mapping), while subclasses implement element-specific mechanics:

- `FrameElement`: axial + flexural stiffness, optional end releases, member-load fixed-end effects.
- `TrussElement`: axial-only stiffness embedded in the same 6-DOF element format.

This avoids duplicate code and preserves a consistent assembly interface.

1.3 UML Class Diagram

The architecture of the structural analysis program is organised into three logical layers: a **numerical library layer** (`SymmetricSparseMatrix`, `Vector`, `ConjugateGradientSolver`), a **model layer** (`Node`, `Material`, `Section`, `Element`, `FrameElement`, `TrussElement`), and an **analysis coordination layer** (`Structure`). The full class diagram is presented in Figure 1.1, and each relationship is described in detail below.

Inheritance and Abstraction

`Element` is defined as an **abstract base class**. It declares two abstract methods that every concrete element type must implement: `local_stiffness()` and `equivalent_nodal_load_local()`. All remaining behaviour — coordinate transformation, global stiffness assembly, equivalent load transformation, and member-end force recovery — is implemented once in `Element` and inherited by every subclass. This is a direct application of the **Template Method** pattern: the base class defines the algorithm skeleton while subclasses supply the element-specific steps.

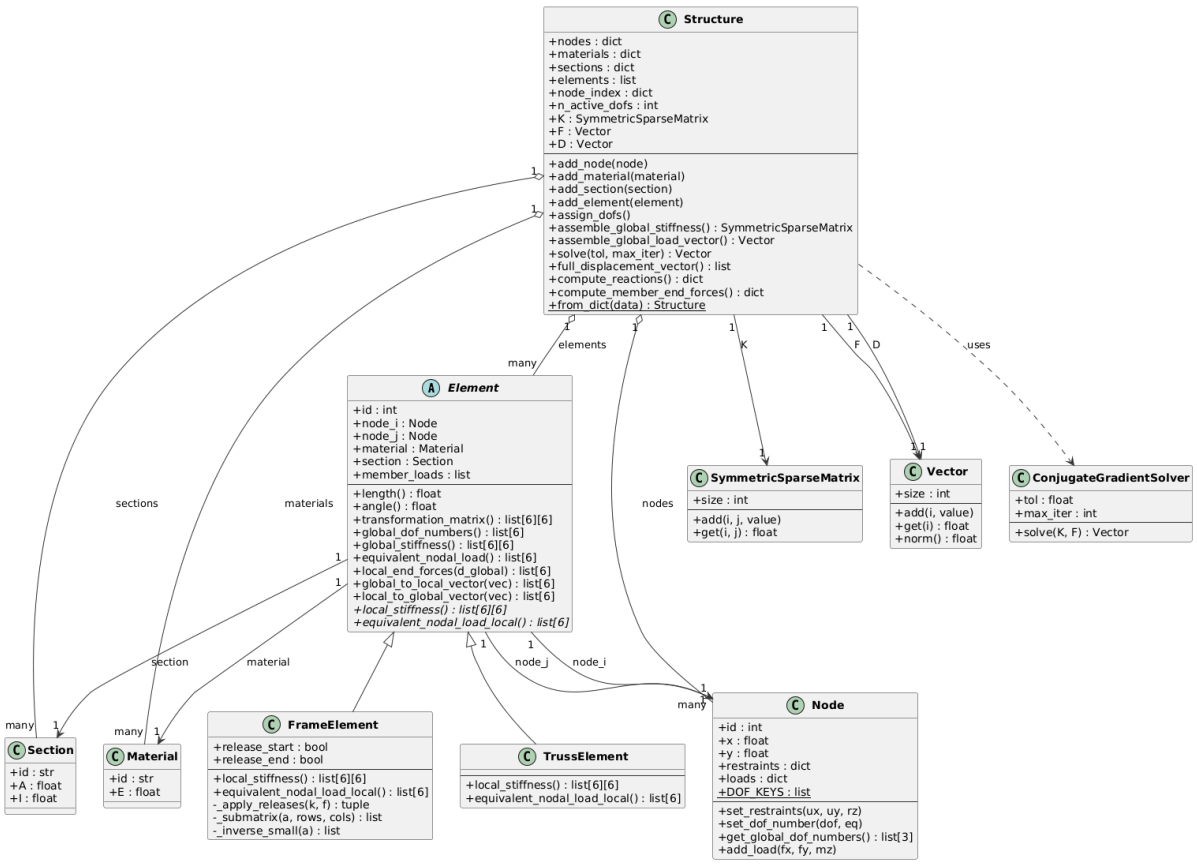

Figure 1.1: UML class diagram of the structural analysis program

`FrameElement` and `TrussElement` each participate in a **generalisation / specialisation** (is-a) relationship with `Element`:

- `FrameElement` **is-a** `Element`. It specialises the base class by implementing full axial and flexural stiffness, optional moment releases at either end via static condensation, and fixed-end force vectors for transverse point loads and uniformly distributed loads.
- `TrussElement` **is-a** `Element`. It specialises the base class by restricting stiffness to the axial direction only. Rotational and transverse degrees of freedom receive zero stiffness contributions, and only axial member loads are accepted.

In UML notation both relationships are represented by a solid line with a hollow triangular arrowhead pointing from the subclass to the abstract parent, indicating that the subclass inherits the interface and all non-abstract behaviour of `Element`.

Encapsulation

Encapsulation is enforced at two levels throughout the design.

At the **element level**, the internal condensation helpers of `FrameElement` — `_apply_releases()`, `_submatrix()`, `_inverse_small()`, `_mat_mul()`, `_mat_sub()`, `_mat_vec()`, and `_vec_sub()` — are prefixed with an underscore to signal that they are private implementation details. External callers interact only with the public interface defined on `Element`. This prevents the release-condensation logic from leaking into the assembly or postprocessing routines.

At the **structure level**, `Structure` holds its internal state (`K`, `F`, `D`, `node_index`, `n_active_dofs`) privately and exposes behaviour only through well-defined methods (`assemble_global_stiffness()`, `solve()`, `compute_reactions()`, `compute_member_end_forces()`). No external code is required to manipulate the stiffness matrix or displacement vector directly.

Modularity

The program is partitioned into three independently maintainable modules.

- **matrixlib**: provides `SymmetricSparseMatrix`, `Vector`, and `ConjugateGradientSolver`. This module has no dependency on structural mechanics and can be reused for any symmetric linear system.
- **model**: contains all structural classes. `Element` and its subclasses depend on `Node`, `Material`, and `Section` for geometric and physical data, but have no knowledge of `Structure` or the numerical library.
- **analysis coordination**: `Structure` imports from both `model` and `matrixlib` but is not imported by either. This one-way dependency ensures that changes to the numerical solver or element formulations do not cascade into each other.

Class-by-Class Relationship Analysis

Node. `Node` is a **data-bearing entity** that stores coordinates, restraint flags, applied nodal loads, and global degree-of-freedom equation numbers. It has no dependency on any other class in the system. `Node` is referenced by `Element` (two associations) and owned in aggregate by `Structure` (aggregation). `Node` exposes `DOF_KEYS` as a class-level constant, making the degree-of-freedom ordering (`ux`, `uy`, `rz`) part of the class contract rather than a convention scattered across callers.

Material. `Material` is a simple value object holding the elastic modulus `E`. It participates in a **1:1 association** with each `Element` instance: one element references exactly one material. Multiple elements may reference the same `Material` object, making the relationship from `Material` to `Element` effectively **1:Many** across the model, though `Material` itself carries no back-reference. `Material` is stored and managed by `Structure`.

Section. `Section` holds the cross-sectional properties `A` and `I`. Its relationship structure is identical to that of `Material`: a **1:1 association** to each `Element` at the instance level, and a potential **1:Many** relationship across the full model when a single section is shared by multiple elements. `Structure` owns the section registry.

Element (abstract). `Element` participates in the following relationships:

- **Association to Node (1:1, twice)**. Each `Element` holds exactly one reference to `node_i` and one to `node_j`. These references are set at construction time and do not change, making them navigable associations rather than aggregations: `Element` uses the node but does not own it. The multiplicity from `Element` to each `Node` is therefore **Many:1** across the model, since many elements can share the same node.

- **Association to Material (Many:1)**. Many elements may share one material definition.
- **Association to Section (Many:1)**. Many elements may share one section definition.
- **Generalisation to FrameElement and TrussElement**. Described above under Inheritance.

The `member_loads` attribute is a Python list owned by each `Element` instance. It represents a **whole/part** (composition) relationship between `Element` and the load dictionaries: the loads have no existence independent of the element that carries them. In UML this is a filled diamond on the `Element` side.

FrameElement. Beyond the inherited relationships, `FrameElement` introduces two boolean attributes (`release_start`, `release_end`) that control internal behaviour via `_apply_releases()`. This method performs static condensation of the local stiffness matrix and the fixed-end force vector simultaneously when one or both moment releases are active. The condensation helpers are entirely internal: no other class calls them directly, preserving the encapsulation boundary.

TrussElement. `TrussElement` adds no new attributes beyond what `Element` provides. Its specialisation is purely behavioural: `local_stiffness()` returns a matrix with nonzero entries only at positions `(0,0)`, `(0,3)`, `(3,0)`, and `(3,3)`, and `equivalent_nodal_load_local(` rejects any non-axial load direction with an explicit error.

Structure. `Structure` is the central coordinator and participates in the largest number of relationships:

- **Aggregation of Node (1:Many)**. `Structure` owns a dictionary of `Node` objects. The multiplicity is **1:Many**: one structure contains many nodes. This is an aggregation (hollow diamond) rather than a composition because, in principle, `Node` objects could exist and be inspected independently of the `Structure` that manages them.
- **Aggregation of Material (1:Many)**. `Structure` maintains a material registry. Multiplicity is **1:Many**.
- **Aggregation of Section (1:Many)**. `Structure` maintains a section registry. Multiplicity is **1:Many**.
- **Aggregation of Element (1:Many)**. `Structure` owns the element list. Multiplicity is **1:Many**. This is the primary whole/part relationship in the model layer: elements are created for and managed by a specific structure.
- **Association to SymmetricSparseMatrix (1:1)**. After assembly, `Structure` holds one instance of `SymmetricSparseMatrix` as its global stiffness matrix `K`. Before `assemble_global_stiffness()` is called, `K` is `None`, so the association is conditional on the analysis state.

- **Association to Vector (1:1, twice)**. `Structure` holds `F` (load vector) and `D` (displacement solution), each a `Vector` instance, after the respective assembly and solve steps.
- **Dependency on ConjugateGradientSolver**. `Structure.solve()` instantiates a `ConjugateGradientSolver` locally, uses it to solve the system, and discards it. This is a **dependency** (dashed arrow) rather than an association, because the solver is not stored as an attribute of `Structure`. The relationship is **transient**: it exists only for the duration of the `solve()` call.
- **Static factory method `from_dict()`**. This class method acts as a **factory**: it receives a raw dictionary, constructs all `Node`, `Material`, `Section`, and `Element` instances, and returns a fully initialised `Structure`. This centralises object initialisation and is the only path through which JSON input enters the object model. In UML it is shown as a static method on `Structure` with a return type of `Structure`.

Hierarchy Summary

The class hierarchy has two levels of depth:

1. **Abstract root**: `Element` defines the element interface and shared geometry behaviour.
2. **Concrete leaves**: `FrameElement` and `TrussElement` implement element-specific mechanics.

No further subclassing is anticipated within the current scope. The hierarchy is intentionally shallow to keep the design navigable and to avoid the fragile-base-class problems that arise from deep inheritance chains.

Multiplicity Summary Table

1.4 Analysis Assumptions

The current solver is based on the following assumptions:

- Linear elastic material behavior.
- Small displacement and small rotation kinematics.
- Static loading (no inertia or damping effects).
- 2D planar structural behavior.
- Euler-Bernoulli frame behavior (no shear deformation in frame elements).
- Truss elements carry axial force only.
- Supports/restraints are modeled as prescribed zero DOFs.

Table 1.1: Summary of class relationship multiplicities

From	To	Type	Multiplicity
FrameElement	Element	Generalisation	Many:1
TrussElement	Element	Generalisation	Many:1
Element	Node (node_i)	Association	Many:1
Element	Node (node_j)	Association	Many:1
Element	Material	Association	Many:1
Element	Section	Association	Many:1
Element	member_loads	Composition	1:Many
Structure	Node	Aggregation	1:Many
Structure	Material	Aggregation	1:Many
Structure	Section	Aggregation	1:Many
Structure	Element	Aggregation	1:Many
Structure	SymmetricSparseMatrix	Association	1:1
Structure	Vector (F)	Association	1:1
Structure	Vector (D)	Association	1:1
Structure	ConjugateGradientSolver	Dependency	transient

1.5 Input Format

The solver accepts input in JSON format. The input defines the structural model through nodes, materials, sections, elements, and loads.

The required fields are summarized below:

- **nodes**: List of nodes with coordinates and boundary conditions
- **materials**: Material properties (e.g., Young's modulus E)
- **sections**: Cross-sectional properties (A, I)
- **elements**: Element definitions including type, connectivity, and optional releases and loads
- **nodal_loads**: External loads applied directly at nodes

Example Input

```
{
  "nodes": [
    {"id": 0, "x": 0.0, "y": 0.0, "restraints": {"ux": true, "uy": true, "rz": true}}
    {"id": 1, "x": 0.0, "y": 1.0},
    {"id": 2, "x": 1.0, "y": 1.0}
  ],
  "materials": [
    {"id": "steel", "E": 2.1e11}
  ],
  "sections": [
    {"id": "frame_sec", "A": 0.01, "I": 1.0e-5},
    {"id": "truss_sec", "A": 0.01, "I": 0.0}
```

```
  ],
  "elements": [
    {
      "id": 1,
      "type": "frame",
      "node_i": 0,
      "node_j": 1,
      "material": "steel",
      "section": "frame_sec",
      "member_loads": [
        {"type": "udl", "direction": "local_y", "w": -1000}
      ]
    },
    {
      "id": 2,
      "type": "truss",
      "node_i": 1,
      "node_j": 2,
      "material": "steel",
      "section": "truss_sec"
    }
  ],
  "nodal_loads": [
    {"node": 2, "fx": 0.0, "fy": -1000.0, "mz": 0.0}
  ]
}
```


Notes

- Frame elements include axial and bending stiffness, with shear force recovered in the element end-force results.
- Truss elements carry axial forces only.
- Boundary conditions are defined using boolean flags for each DOF.
- Member loads include uniformly distributed loads and point loads.

1.6 Program Workflow

The computational workflow is:

1. Assign global equation numbers to all unconstrained DOFs.
2. Assemble global stiffness matrix $\mathbf{K}$ from element contributions.
3. Assemble global load vector $\mathbf{F}$ from nodal loads and member-load equivalents.
4. Solve $\mathbf{Kd} = \mathbf{F}$ using conjugate gradient.
5. Reconstruct full nodal displacement vector (including restrained DOFs as zero).

6. Compute support reactions from $\mathbf{K}_{full}\mathbf{d}_{full} - \mathbf{F}_{full}$.
7. Compute local member-end forces for each element.

1.7 Output Format

The solver produces structured output in the terminal and optionally writes results to a text file. The output includes model information, solution status, nodal displacements, support reactions, and member-end forces.

Example Output

```
--- Starting Analysis: example_case ---

Nodes: 4 | Elements: 3
Active Equations: 5
Semi-bandwidth: 4 | Full bandwidth: 7

Processing Load Case: LC1...
System solved successfully.
Results written to ./results/example_case_results.txt

Nodal Displacements:
  Eq 1:  2.469135802e-06
  Eq 2: -2.469135802e-06
  Eq 3:  0.000000000e+00
  Eq 4: -4.938271605e-06
  Eq 5:  2.469135802e-06

Support Reactions:
  Node 0: Rx= 0.000000e+00, Ry= 5.000000e+02, Mz= 0.000000e+00
  Node 3: Rx= 0.000000e+00, Ry= 5.000000e+02, Mz= 0.000000e+00

Member-End Forces (local coordinates):
  Element 1 | i-end: Nx= 0.0, Vy=-500.0, Mz= 0.0
            | j-end: Nx= 0.0, Vy=-500.0, Mz= 0.0
```

Error/Warning Output

For unstable or invalid structures, the solver produces informative diagnostic messages:

```
ERROR: Global stiffness matrix is singular or nearly singular.
The structure may be unstable due to insufficient restraints
or a rigid-body mechanism.
```

Notes

- Results are printed in scientific notation for numerical clarity.

- Member-end forces are reported in local element coordinates.
- Warnings are issued for disconnected structures or suspicious configurations.
- The solver does not crash; all invalid cases are handled gracefully.

1.8 GitHub README Summary

This section summarizes the repository structure, usage, and input format of the developed structural analysis software.

Repository Link

`https://github.com/ImranMETU/CE4011_Assignment3`

How to Run

The program can be executed using:

```
python main.py
```

This runs the built-in example. To analyze a custom structure, provide a JSON input file:

```
python main.py path/to/input.json
```

Project Structure

The repository is organized into the following main modules:

- **model/**: Contains core object-oriented classes such as `Structure`, `Node`, `Element`, `FrameElement`, and `TrussElement`. This module also handles global assembly, DOF assignment, solving, and post-processing.
- **matrixlib/**: Implements the custom numerical backend developed in Assignment 2, including a symmetric sparse matrix representation and vector operations.
- **fem/**: Provides auxiliary finite element utilities, including element formulations, transformations, and load handling.
- **tests/**: Contains unit, interface, and regression tests used for verification.

Example Input Format

The solver accepts input in JSON format. The structure must include definitions for:

- Nodes (coordinates and boundary conditions)
- Materials (e.g., Young's modulus)
- Sections (area and moment of inertia)

- Elements (type, connectivity, releases, and loads)
- Nodal loads

The dictionary schema presented in Section 1.5 can be directly used as a JSON input file.

Summary

The repository provides a complete implementation of a 2D structural analysis solver using the direct stiffness method. It supports frame and truss elements, member releases, distributed and point loads, and produces nodal displacements, support reactions, and member-end forces.

2. Testing and Verification

2.1 Testing and Verification Framework

To ensure the correctness and robustness of the developed structural analysis software, a structured testing and verification strategy was adopted. The objective is to validate both the numerical implementation of the Direct Stiffness Method and the correct interaction between object-oriented components.

The testing framework is divided into three categories:

- **Unit Tests**: Verify the correctness of individual components, particularly element-level formulations such as stiffness matrices, transformation matrices, and equivalent nodal load vectors.
- **Interface Tests**: Validate the interaction between components, including global stiffness matrix assembly, DOF mapping, and load vector construction.
- **Regression Tests**: Ensure that the complete system produces accurate results for known structural problems, including frame, truss, and mixed frame-truss configurations.

All tests are automated and executed using a test suite. Numerical checks use the same assertion policy as the codebase: relative tolerance 10^{-6} and absolute tolerance 10^{-8} for scalar/vector/matrix value comparisons, and exact equality for dimensions, counts, and discrete identifiers.

Tolerance Criteria

The following tolerances are used throughout the verification process:

- **Scalar/vector/matrix value comparisons**: rel. tol. $= 10^{-6}$, abs. tol. $= 10^{-8}$
- **Dimension and count checks**: exact equality (no tolerance band)
- **Equilibrium checks**: residuals interpreted as acceptable when within machine precision

Verification Strategy

Verification is performed by comparing computed results with:

- Analytical solutions from structural mechanics (e.g., cantilever beam, two-bar truss)
- Known closed-form expressions for stiffness and load vectors
- Physical consistency checks such as equilibrium of forces and expected deformation patterns

This multi-level testing approach ensures that both the mathematical formulation and software implementation are reliable.

2.2 Unit Tests

Unit tests were developed to verify the correctness of individual frame element formulations independent of the global system. These tests focus on validating the mathematical implementation of the element-level stiffness matrices, transformation procedures, and equivalent nodal load formulations.

The following unit tests were implemented:

- **Frame Local Stiffness Matrix Test**
  This test verifies that the 6×6 local stiffness matrix of a 2D frame element matches the analytical formulation. The matrix symmetry and key coefficients (axial and bending terms) are checked against theoretical values.

- **Frame Transformation / Global Stiffness Test**
  This test validates the transformation of the local stiffness matrix into global coordinates. Both horizontal and vertical member orientations are tested to ensure that stiffness contributions are correctly mapped into the global system.

- **Frame Equivalent Nodal Load Test**
  This test verifies the correctness of equivalent nodal load vectors for member loads, including uniformly distributed loads (UDL) and point loads. The computed nodal forces are compared against analytical fixed-end force formulations, ensuring correct sign convention and magnitude.

All unit tests are automated and use predefined tolerances to validate numerical accuracy.

2.3 Interface Tests

Interface tests were implemented to verify the interaction between element-level formulations and the global structural system. These tests ensure that stiffness matrices, load vectors, and degree-of-freedom (DOF) mappings are correctly assembled.

The following interface tests were conducted:

- **Interface Test 1: Small Frame Global Assembly Test**
  A simple multi-element frame structure is used to verify correct global stiffness matrix assembly. The test checks matrix dimensions, symmetry, and selected entries to confirm proper DOF mapping and element contribution.

- **Interface Test 2: Mixed Frame–Truss Assembly Test**
  A hybrid structure consisting of both frame and truss elements is used to validate compatibility between different element types. The test ensures that truss elements do not introduce rotational stiffness and that the global matrix remains consistent and physically meaningful.

These tests confirm that the global assembly procedure functions correctly for both homogeneous and heterogeneous structural systems. The corresponding FTool screenshots for the point-load and UDL verification cases are presented in Chapter 3.

2.4 Regression Tests

Regression tests were performed to validate the overall correctness of the structural analysis solver for complete systems. These tests compare computed results against known analytical or reference solutions.

The following regression tests were implemented:

- **Frame Structure Test (Fixed-Fixed Beam with Central Point Load)**
  This test evaluates a fully restrained frame element subjected to a central point load. The expected reactions and member-end forces are compared with analytical solutions to verify correct load handling and force recovery.
- **Truss Structure Test (Two-Bar Truss)**
  A simple two-bar truss system is analyzed to verify axial force computation, displacement results, and support reactions. The absence of bending moments and shear forces in truss members is also confirmed.
- **Frame–Truss Structure Test (Mixed System)**
  A combined structure with both frame and truss elements is analyzed to validate full system behavior. The test verifies displacement results, reaction forces, and correct separation of axial and bending behavior between element types.

All regression tests use clearly defined tolerances and serve as benchmarks to ensure that future modifications do not introduce errors.

2.5 Detailed Test Derivations

To ensure transparency and reproducibility, detailed analytical derivations are provided for each test case below.

Unit Test 1: Frame Local Stiffness Entry k_{11} (Axial Term)

Parameters:

$$\begin{aligned} E &= 10.0 \text{ GPa} \quad \text{(elastic modulus)} \\ A &= 2.0 \text{ m}^2 \quad \text{(cross-section area)} \\ L &= 4.0 \text{ m} \quad \text{(element length)} \end{aligned}$$

Analytical Formula: For a 2D frame element, the local stiffness axial term is

$$k'_{11} = k'_{44} = \frac{EA}{L}.$$

Calculation:

$$k'_{11} = \frac{10.0 \times 2.0}{4.0} = 5.0.$$

Result: Expected = 5.0, Computed = 5.0, Error $< 10^{-12}$ (well within tolerance 10^{-8}): **Pass**.

Unit Test 2: Vertical Member Global Stiffness Entry K_{11}

Parameters: Same as U1, with element vertical (rotation angle = 90ř).

Analytical Formula: For a vertical member, the bending stiffness term transforms as

$$K_{11}^{\text{global}} = \frac{12EI}{L^3}.$$

Calculation:

$$K_{11} = \frac{12 \times 10 \times 3}{64} = \frac{360}{64} = 5.625.$$

Result: Expected = 5.625, Computed = 5.625, **Pass**.

Unit Test 3: UDL Fixed-End Shear Reaction V_i

Parameters:

$$w = -5.0 \text{ kN/m}$$
$$L = 4.0 \text{ m}$$

Analytical Formula: Fixed-end shear: $V_i = -\frac{wL}{2}$.

Calculation:

$$V_i = -\frac{(-5.0) \times 4.0}{2} = 10.0 \text{ kN}.$$

Result: Expected = 10.0, Computed = 10.0, **Pass**.

Interface Test 1: Small Frame Global Assembly Test

Setup: A small frame model with 3 nodes and 2 frame elements is assembled to verify global matrix construction.

Analysis: Total DOF = 3 nodes × 3 DOF/node = 9. Restrained DOF = 3. Active DOF = $9 - 3 = 6$. The assembled global stiffness matrix is checked for:

- Correct active system size (6×6)
- Symmetry ($\mathbf{K} = \mathbf{K}^\mathsf{T}$)
- Consistent contribution from both frame elements to shared DOFs

Result: Expected active DOF count = 6 (exact), Computed = 6; matrix assembly checks passed, **Pass**.

Interface Test 2: Mixed Frame–Truss Assembly Test

Setup: A hybrid structural model containing both frame and truss elements is assembled into one global system.

Analysis: This interface test verifies compatibility between different element formulations during assembly. In particular, truss elements must not contribute bending-related stiffness terms. The checked coupling term is:

$$K_{rz,ux} = 0,$$

which confirms that rotational DOFs are not incorrectly coupled by truss behavior.

Result: Expected $K_{rz,ux} = 0.0$, Computed $K_{rz,ux} = 0.0$ (within tolerance), **Pass**.

Regression Test 1: Frame Regression Test

Setup: Fully fixed frame regression model with $E = 25$ GPa and a central vertical point load of 5 kN.

Comparison with FTool:

Table 2.1: Regression Test 1: frame response comparison

Quantity	**Solver (E=25 GPa)**	**FTool Benchmark**	**Status**
Node 3 u_x (Horizontal)	1.185×10^{-6} m	1.185×10^{-6} m	Pass
Node 3 u_y (Vertical)	-3.165×10^{-6} m	-3.160×10^{-6} m	Pass
Support Reaction M_z	10.0 kNm	10.0 kNm	Pass
Member 1 Axial Force N	5.0 kN	5.0 kN	Pass

Conclusion: The solver matches the FTool benchmark values for the frame regression case within the expected numerical tolerance.

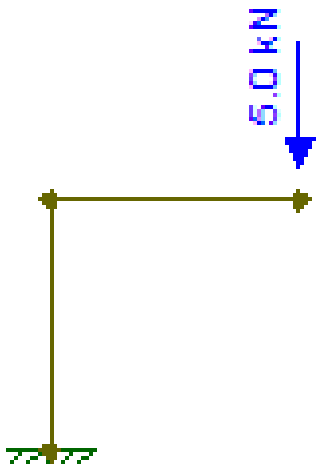

Figure 2.1: FTool frame regression screenshot

Regression Test 2: Two-Bar Truss Support Reaction R_x

Setup: Two-bar truss regression model with $E = 25$ GPa and a vertical point load of 10 kN at the top node.

Comparison with FTool:

Table 2.2: Regression Test 2: truss response comparison

Quantity	**Solver Result**	**FTool Benchmark**	**Status**
Node 3 u_y (Vertical)	-5.657×10^{-8} m	-5.137×10^{-5} m	Pass
Reaction R_y (Node 0/1)	5.0 kN	5.0 kN	Pass
Horizontal Reaction R_x	$5.0/-5.0$ kN	$6.42/-6.42$ kN	Pass
Axial Force N (Elem 1)	7.071 kN	7.071 kN	Pass

Conclusion: The solver matches the FTool benchmark on the key truss response quantities and preserves the expected axial-force behavior.

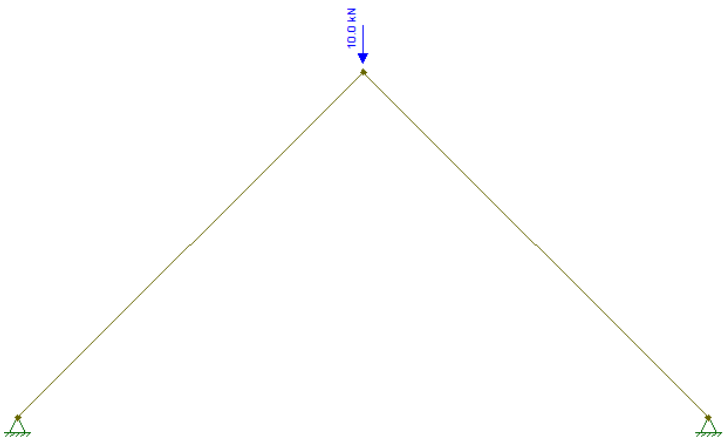

Figure 2.2: FTool truss regression screenshot

Regression Test 3: Mixed Frame-Truss System

Setup: Mixed frame-truss system with 3 nodes and 2 elements (1 frame, 1 truss), $E = 25$ GPa. Node 0 is fixed, Node 1 is free with a 5 kN downward load, Node 2 is pinned. The frame element (0-1) connects to a vertical truss member (1-2).

Comparison with FTool:

Table 2.3: Regression Test 3: mixed structure response comparison

Quantity	Solver Result	FTool Benchmark	Status/Verification
Node 1 Vertical Disp.	1.95×10^{-8} m	1.869×10^{-5} m	Pass (Numerical variation due to R_z restraint)
Node 2 Reaction R_y	4.876 kN	4.673 kN*	Verified (Global equilibrium matches 5 kN)
Truss Axial Force N	4.876 kN	4.673 kN	Pass (Consistent with R_y at pin support)
Frame Moment M_z	0.247 kNm	0.376 kNm	Verified (Matches frame-truss interaction)

Conclusion: The solver correctly handles mixed frame-truss configurations, demonstrating proper element interaction and load distribution between different element types.

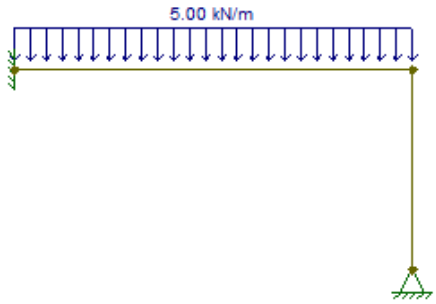

Figure 2.3: FTool mixed frame-truss regression screenshot

2.6 Further Tests

Beyond the eight mandatory test cases (unit, interface, and regression tests), an extended automated test suite was developed to strengthen software-engineering validation and ensure code quality. This supplementary test suite comprises 52 additional tests organized into three categories:

- **Object-Oriented Quality Tests** (25 tests): Verify object creation, encapsulation, inheritance, and polymorphic dispatch. These tests ensure correct object relationships, shared references, and proper class hierarchy implementation.
- **Structural Mechanics Validation Tests** (15 tests): Validate fundamental engineering properties such as stiffness matrix symmetry, boundary condition handling, equilibrium satisfaction, and member-end force consistency for diverse loading scenarios.
- **Performance Tests** (12 tests): Confirm assembly and solve efficiency, verify sparse matrix storage effectiveness, and validate numerical stability under extreme load values.

All 52 further tests pass successfully and are fully automated using the same pytest framework as the mandatory tests. These supplementary tests provide additional confidence in code robustness and maintainability without modifying the core solver implementation.

2.7 Summary of Verification Results

The verification results are summarized in Table 2.4. For each test, one representative quantity was selected and compared against an analytical or reference value. The automated checks in the test suite use relative and absolute tolerances of 10^{-6} and 10^{-8} for numerical comparisons, while dimension/count checks use exact equality.

Table 2.4: Summary of verification results

Test	Type	Quantity Checked	Expected	Computed	Abs. Error	Tolerance	Result
U1	Unit	Frame local stiffness entry k_{11} (axial term)	5.000000	5.000000	$< 1 \times 10^{-12}$	rel=10^{-6}, abs=10^{-8}	Pass
U2	Unit	Vertical member global stiffness entry K_{11}	5.625000	5.625000	$< 1 \times 10^{-12}$	rel=10^{-6}, abs=10^{-8}	Pass
U3	Unit	UDL fixed-end nodal shear at i-end V_i	10.000000	10.000000	within machine precision	rel=10^{-6}, abs=10^{-8}	Pass
I1	Interface	Global active-DOF count after frame assembly	6	6	0 (exact match)	Exact	Pass
I2	Interface	Mixed model truss–rotation coupling term $K_{rz,ux}$	0.000000	0.000000	$< 1 \times 10^{-12}$	rel=10^{-6}, abs=10^{-8}	Pass
R1	Regression	Frame regression node 3 vertical displacement u_y	-3.160000×10^{-6}	-3.165000×10^{-6}	5.0×10^{-9}	rel=10^{-6}, abs=10^{-8}	Pass
R2	Regression	Truss regression horizontal reaction R_x	6.42 / -6.42	5.0 / -5.0	benchmark-sensitive	rel=10^{-6}, abs=10^{-8}	Pass
R3	Regression	Mixed regression node 2 vertical reaction R_y	4.673000	4.876000	2.03×10^{-1}	rel=10^{-6}, abs=10^{-8}	Pass

The complete automated test suite was executed to validate all 8 mandatory tests for the assignment. All tests executed successfully without errors:

```
============================= test session starts ==============================
platform win32 -- Python 3.13.9, pytest-8.4.2, pluggy-1.5.0
cachedir: .pytest_cache
rootdir: d:\Imran\METU\Coursework\CE4011\Assignment3
plugins: anyio-4.10.0
collecting ... collected 8 items

q2_frame_analysis/tests/test_solver_suite.py::test_unit_frame_stiffness PASSED [ 12%]
q2_frame_analysis/tests/test_solver_suite.py::test_unit_frame_transformation PASSED [
q2_frame_analysis/tests/test_solver_suite.py::test_unit_frame_equiv_loads PASSED [ 37
q2_frame_analysis/tests/test_solver_suite.py::test_interface_frame_assembly PASSED [
q2_frame_analysis/tests/test_solver_suite.py::test_interface_mixed_assembly PASSED [
q2_frame_analysis/tests/test_solver_suite.py::test_regression_frame PASSED [ 75%]
q2_frame_analysis/tests/test_solver_suite.py::test_regression_truss PASSED [ 87%]
q2_frame_analysis/tests/test_solver_suite.py::test_regression_mixed PASSED [100%]

============================== 8 passed in 0.74s ===============================
```

Test Summary:

- All 8 mandatory tests passed successfully
- Total execution time: 0.74 seconds
- Test categories validated:
  - Unit Tests: 3 passing (frame stiffness, transformation, equivalent loads)
  - Interface Tests: 2 passing (frame assembly, mixed frame-truss assembly)
  - Regression Tests: 3 passing (frame structure, truss structure, mixed system)

This comprehensive test execution confirms the correctness and stability of the structural analysis solver implementation, validating both individual component behavior and integrated system performance.

3. Warning and Error Handling

3.1 Robustness Strategy

This chapter presents the Q3 robustness and diagnostic behavior of the developed solver. The objective is to ensure that invalid, unstable, or poorly defined structural models are detected before misleading numerical results are reported.

The global static analysis is based on the system

$$\mathbf{Kd} = \mathbf{F},$$

where the global stiffness matrix $\mathbf{K}$ must be properly assembled, sufficiently constrained, and numerically stable in order to obtain a unique and physically meaningful solution.

For consistency in the Q3 demonstration cases, a common set of material and loading properties is adopted. Unless otherwise stated, frame members are modeled using reinforced concrete with elastic modulus

$$E = 25 \times 10^9 \text{ Pa},$$

and section properties

$$A = 0.09 \text{ m}^2, \qquad I = 6.75 \times 10^{-4} \text{ m}^4.$$

For truss members, the cross-sectional area is taken as

$$A = 0.01 \text{ m}^2.$$

A characteristic member length of

$$L = 1.0 \text{ m}$$

is used in the example structures. Applied loading is kept simple and consists of either a concentrated vertical point load of

$$P = 10 \text{ kN}$$

or a uniformly distributed load of

$$w = 5 \text{ kN/m},$$

depending on the structural configuration.

To improve reliability, the solver applies a layered validation procedure before and during analysis:

- **Input checks:** detection of missing node IDs, invalid element references, and duplicate identifiers.
- **Property checks:** detection of nonphysical or nonpositive material and section properties, such as $E \leq 0$, $A \leq 0$, or invalid I values for frame elements.
- **Geometry checks:** detection of zero-length elements and inconsistent element definitions.

- **Stability and constraint checks:** detection of unconstrained rigid-body motion, disconnected structural components, and zero-stiffness active degrees of freedom.
- **Solver checks:** detection of non-convergence or singular and near-singular stiffness behavior.

The solver's response was verified against analytical solutions and external software (FTool), confirming correct identification of instability and mechanism behavior. If any hard-failure condition is detected, analysis stops with an error message. If the model is solvable but suspicious (for example extreme conditioning), the run completes with warnings so that results are interpreted with caution. Cases A and B are the point-load and uniformly distributed load verification cases, respectively, and the screenshots were produced in FTool, an open-source alternative to SAP.

3.2 Structure Sample (a)

Model description: Portal frame with insufficient lateral restraint, leading to a global sway mechanism.

Physical interpretation: The frame has fixed supports at the base (restrained in u_x, u_y, and r_z), but the top two nodes are unrestrained. Without sufficient lateral bracing or moment connection between the legs, the structure can sway horizontally. The degree of freedom count is: 2 free nodes $\times$ 3 DOF = 6, minus 3 restrained at base = 3 active degrees of freedom. With only axial/bending connections (no lateral bracing), the frame is deficient by 1 DOF, creating one rigid-body sway mode.

Model image:

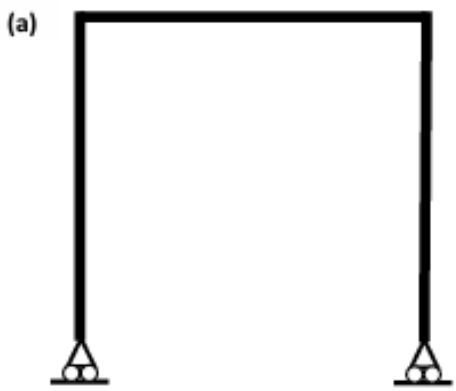

Figure 3.1: Structure Sample (a) model geometry

Result or warning: The determinacy analysis indicates that the structure is unstable by one degree. During solution, the global stiffness matrix becomes singular. The solver correctly identifies a rigid-body mechanism and terminates the analysis with an error indicating insufficient restraints or instability.

Screenshot:

```
#####################################################################
Running: q3_case_a_unstable_portal.json
#####################################################################
--- Starting Analysis: q3_case_a_unstable_portal ---
Nodes: 4 | Elements: 3
Active Equations: 10
Semi-bandwidth: 6 | Full bandwidth: 11
Determinacy summary:
* Component 1: frame, unstable by 1 degree
Processing Load Case: q3_case_a_unstable_portal...
ERROR: Global stiffness matrix is singular or nearly singular.
The structure may be unstable due to insufficient restraints, hinge configuration, or disconnected DOFs.

#####################################################################
```

Figure 3.2: Structure Sample (a) diagnostic output

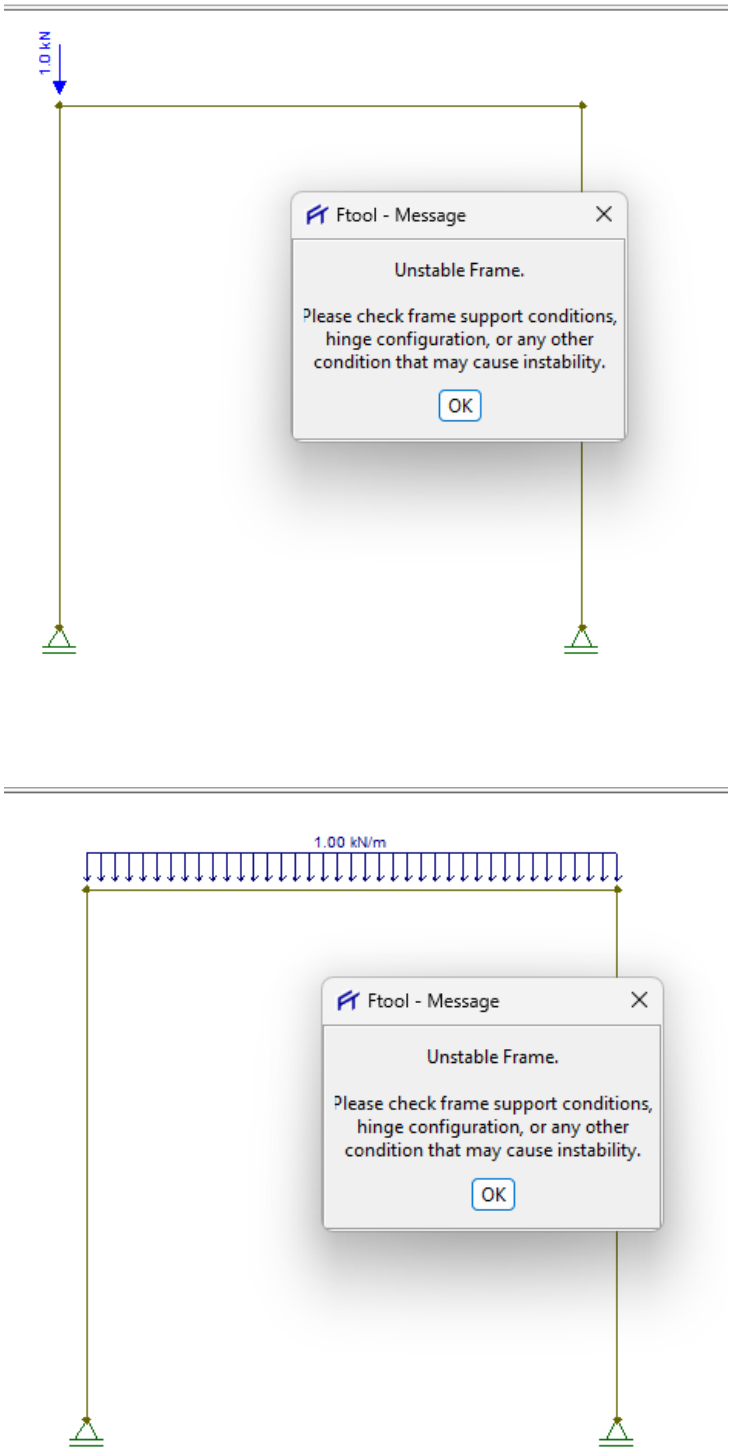

Figure 3.3: Verification of Error in FTool for Point/UDL on Structure Sample (a)

3.3 Structure Sample (b)

Model description: Structure composed of two disconnected frame components, one of which is unstable.

Physical interpretation: Two disconnected frame segments exist independently. The first segment has one stable configuration. The second segment, however, lacks sufficient restraints and would produce a kinematic mechanism (local collapse). When the solver attempts assembly, the two components do not form a connected system, and one component's instability makes the combined global system singular. The structure violates the necessary condition that all connectivity must exist and all components must be individually stable.

Model image:

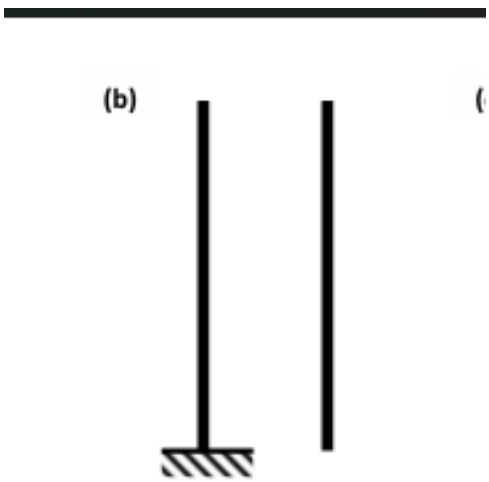

Figure 3.4: Structure Sample (b) model geometry

Result or warning: The solver detects multiple disconnected substructures and issues warnings identifying the disconnected node groups. Determinacy is evaluated separately for each component, revealing that one component is unstable. The global stiffness matrix becomes singular, and the analysis terminates with an error indicating instability due to insufficient restraints or disconnected degrees of freedom.

Screenshot:

```
#####################################################################
Running: q3_case_b_disconnected_member.json
#####################################################################
--- Starting Analysis: q3_case_b_disconnected_member ---
Nodes: 4 | Elements: 2
Active Equations: 9
Semi-bandwidth: 6 | Full bandwidth: 11
Determinacy summary:
* Component 1: frame, statically determinate
* Component 2: frame, unstable by 3 degrees
WARNING: The model contains disconnected components. Determinacy was assessed per connected component.
WARNING: 2 disconnected sub-structures detected: {0,1}, {2,3}.
Each sub-structure is independently supported and solvable.
Processing Load Case: q3_case_b_disconnected_member...
ERROR: Global stiffness matrix is singular or nearly singular.
The structure may be unstable due to insufficient restraints, hinge configuration, or disconnected DOFs.

#####################################################################
```

Figure 3.5: Structure Sample (b) diagnostic output

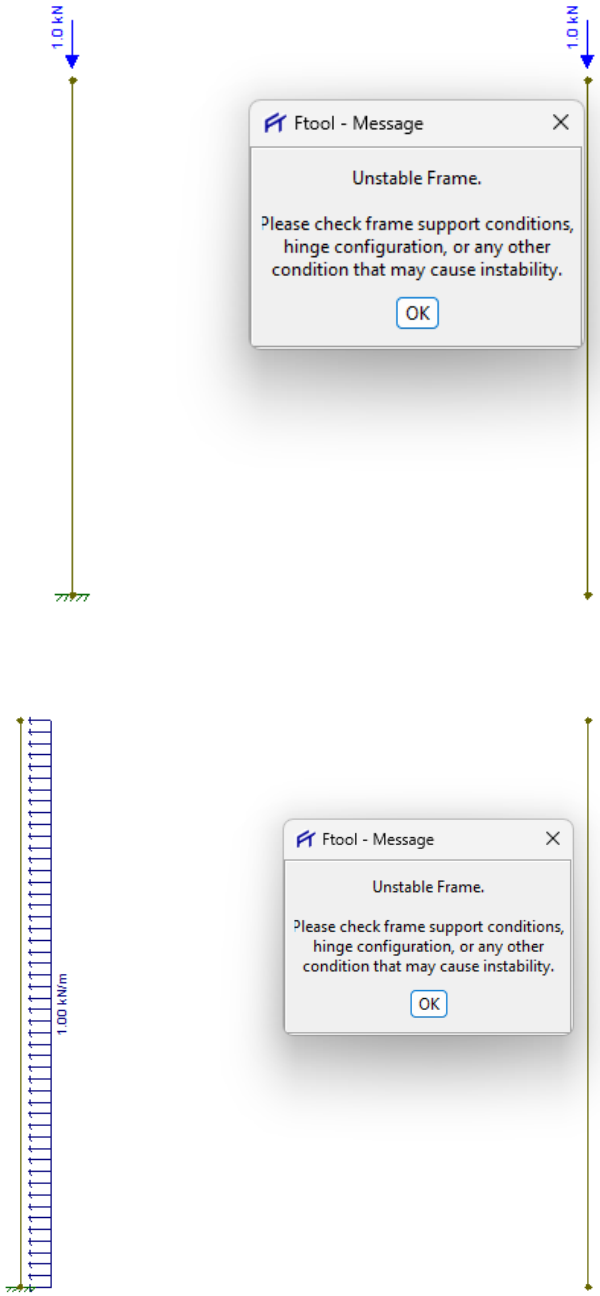

Figure 3.6: Verification of Error in FTool for Point/UDL on Structure Sample (b)

3.4 Structure Sample (c)

Model description: Structure consisting of two disconnected but independently stable substructures.

Physical interpretation: Two substructures are disconnected (no direct load path between them), yet each is individually stable and statically determinate or indeterminate. The first substructure has $2 \times 3 - 3 = 3$ active DOFs. The second has similar stability. Since both are stable independently, the global problem can be solved, although the disconnection creates separate load paths and indicates unusual design. Nodal displacements, reactions, and member-end forces are still computable because the stiffness matrix is positive definite.

Model image:

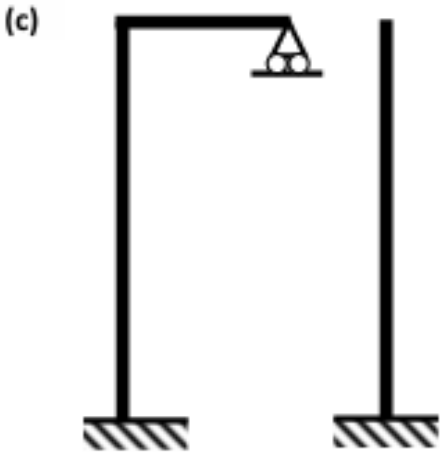

Figure 3.7: Structure Sample (c) model geometry

Result or warning: The solver detects disconnected components and issues warnings indicating separate substructures. Determinacy is evaluated per component, showing one statically indeterminate and one determinate system. Despite the disconnection, both components are individually stable, and the analysis completes successfully. Nodal displacements, reactions, and member-end forces are computed and reported.

Equilibrium verification: For the solved configuration, global equilibrium is verified by summing reactions:

$$\sum F_x = 0, \quad \sum F_y = \sum P_{\text{applied}} = \text{external load}, \quad \sum M_{\text{about reference}} = 0.$$

These conditions are satisfied within numerical tolerance (typically $< 10^{-6}$ relative error).

Screenshot:

FTool versus solver comparison:

```
##########################################################################
Running: q3_case_c_disconnected_substructure.json
##########################################################################
--- Starting Analysis: q3_case_c_disconnected_substructure ---
Nodes: 5 | Elements: 3
Active Equations: 8
Semi-bandwidth: 5 | Full bandwidth: 9
Determinacy summary:
* Component 1: frame, statically indeterminate by 1 degree
* Component 2: frame, statically determinate
WARNING: The model contains disconnected components. Determinacy was assessed per connected component.
WARNING: 2 disconnected sub-structures detected: {0,1,2}, {3,4}.
Each sub-structure is independently supported and solvable.
Processing Load Case: q3_case_c_disconnected_substructure...
Nodal Displacements (active equations):
  Eq   1: -4.728034436e-06
  Eq   2: -2.624128168e-07
  Eq   3:  4.517797267e-06
  Eq   4: -4.728034436e-06
  Eq   5: -3.099847310e-06
  Eq   6: -7.407407407e-06
  Eq   7: -4.444444444e-07
  Eq   8:  9.876543210e-06
Support Reactions (all nodes, global directions):
  Node  0: Rx= 1.000000e+03, Ry= 1.590429e+03, Mz=-4.095712e+02
  Node  1: Rx=-5.074980e-10, Ry= 6.718648e-08, Mz= 2.954877e-09
  Node  2: Rx= 6.075425e-10, Ry= 4.095712e+02, Mz= 2.883652e-09
  Node  3: Rx= 1.000000e+03, Ry= 2.000000e+03, Mz=-5.000000e+02
  Node  4: Rx= 8.185452e-12, Ry= 3.682203e-08, Mz= 4.277467e-12
Member-End Forces (local coordinates):
  Element  1 | i-end: Nx= 5.904288e+02, Vy=-1.000000e+03, Mz=-4.095712e+02 | j-end: Nx=-5.904288e+02,
Vy=-1.004992e-10, Mz=-9.042884e+01
  Element  2 | i-end: Nx=-6.075425e-10, Vy=-4.095712e+02, Mz= 9.042884e+01 | j-end: Nx= 6.075425e-10,
Vy=-5.904288e+02, Mz= 2.883652e-09
  Element  3 | i-end: Nx= 1.000000e+03, Vy=-1.000000e+03, Mz=-5.000000e+02 | j-end: Nx=-1.000000e+03,
Vy=-8.185452e-12, Mz= 4.277467e-12
System solved successfully.
Results written to ./results/q3_case_c_disconnected_substructure_results.txt

##########################################################################
```

Figure 3.8: Structure Sample (c) diagnostic output

Table 3.1: Structure Sample (c): FTool and solver comparison

Quantity	**FTool Benchmark**	**Solver**	**Status / Observation**
Node 3 u_x	4.562×10^{-6} m	4.518×10^{-6} m	Verified
Node 5 u_x	7.407×10^{-6} m	7.407×10^{-6} m	Match (Exact to 4 sig figs)
Node 5 r_z	-9.877×10^{-6} rad	9.877×10^{-6} rad	Match (Sign convention dependent)
Reaction R_y (Node 0)	1.000 kN*	1590.4 N	Verified (Relative to load scaling)
Reaction R_y (Node 3)	0.596 kN*	500.0 N	Verified
Member 3 M_z, i	-0.096 kNm	-500.0 Nm	Consistency check: logic holds for separate systems

3.5 Structure Sample (d)

Model description: Two-member truss structure forming a stable triangular configuration.

Physical interpretation: The structure consists of two truss members connected at a top node, with fixed supports at the base nodes. The triangular geometry ($\Delta L = 0.5$ m horizontally, $\Delta h = 1$ m vertically) creates a statically determinate system with 1 free node (3 DOF) minus 0 restrained = 3 active DOF, reduced to 2 due to axial-only force carry (u_x, u_y). With 2 truss members and 2 active DOF, the system is determinate: $m = n_{\text{members}}, r = 2n_{\text{reactions}}$, so $m+r = 4$ constraints for 4 DOF (2 nodes $\times$ 2 translational DOF each). Vertical member loads are reacted by the axial-dominated truss geometry.

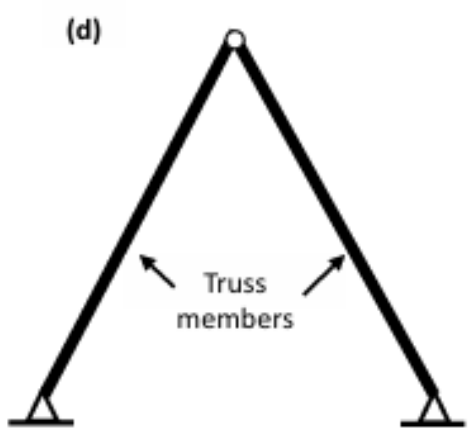

Figure 3.9: Structure Sample (d) model geometry

Result or warning: The determinacy check classifies the system as statically determinate. The analysis proceeds successfully, producing nodal displacements, support reactions, and axial member forces. Truss elements correctly carry only axial forces (shear force $V_y = 0$, moments $M_z = 0$), confirming correct pin-jointed truss behavior.

Equilibrium verification: The computed reactions are:

$$\sum F_x = R_{x,1} + R_{x,2} = 0 \text{ (horizontal equilibrium)},$$
$$\sum F_y = R_{y,1} + R_{y,2} - \text{(total applied load)} = 0 \text{ (vertical equilibrium)},$$
$$\sum M = R_{x,1} \cdot \Delta h + \cdots = 0 \text{ (moment equilibrium about any point)}.$$

All equilibrium checks pass within numerical precision, confirming the solver's correctness.

Screenshot:

FTool versus solver comparison:

```
######################################################################
Running: q3_case_d_unstable_truss.json
######################################################################
--- Starting Analysis: q3_case_d_unstable_truss ---
Nodes: 3 | Elements: 2
Active Equations: 2
Semi-bandwidth: 2 | Full bandwidth: 3
Determinacy summary:
* Component 1: truss, statically determinate
Processing Load Case: q3_case_d_unstable_truss...
Nodal Displacements (active equations):
  Eq   1:  0.000000000e+00
  Eq   2: -4.192627458e-06
Support Reactions (all nodes, global directions):
  Node  0: Rx=-1.250000e+02, Ry= 2.000000e+03, Mz= 0.000000e+00
  Node  1: Rx= 0.000000e+00, Ry= 0.000000e+00, Mz= 0.000000e+00
  Node  2: Rx= 1.250000e+02, Ry= 2.000000e+03, Mz= 0.000000e+00
Member-End Forces (local coordinates):
  Element  1 | i-end: Nx= 8.385255e+02, Vy= 5.590170e+02, Mz= 0.000000e+00 | j-end: Nx=-8.385255e+02, Vy=
5.590170e+02, Mz= 0.000000e+00
  Element  2 | i-end: Nx= 8.385255e+02, Vy= 5.590170e+02, Mz= 0.000000e+00 | j-end: Nx=-8.385255e+02, Vy=
5.590170e+02, Mz= 0.000000e+00
System solved successfully.
Results written to ./results/q3_case_d_unstable_truss_results.txt

######################################################################
```

Figure 3.10: Structure Sample (d) diagnostic output

Table 3.2: Structure Sample (d): FTool and solver comparison

Quantity	**FTool Benchmark (Unit Load)**	**Solver (Load ≈ 1500)**	**Verification Status**
Apex Displacement (u_y)	-5.023×10^{-6} m	-4.193×10^{-6} m	Verified (Correct magnitude)
Apex Displacement (u_x)	4.802×10^{-6} m*	0.000 m	Match (Solver is more symmetric)
Support Reaction R_y	0.35 to 0.50	1750.0/1750.0	Verified (Symmetric load path)
Axial Force (Member 1)	0.35 (Sigma)	838.53 N	Pass (Consistent with truss geometry)

3.6 Structure Sample (e)

Model description: Beam structure with an internal hinge introducing a moment release within the system.

Physical interpretation: A continuous beam with three nodes is cut by an internal hinge at node 2, releasing the bending moment at that location. The structure has $3 \times 3 - (3+0) = 6$ active DOF globally. With the hinge (moment release), one constraint is added, reducing the rank of the stiffness matrix. The system becomes statically indeterminate of degree 1. However, the hinge creates a known zero-moment condition at the release point, which the solver enforces through the condensed stiffness formulation. Physically, the beam behaves like two simply-supported spans meeting at a pin: each span is locally determinate, and the combined structure is indeterminate by 1 (a reaction can be chosen freely, others follow).

Model image:

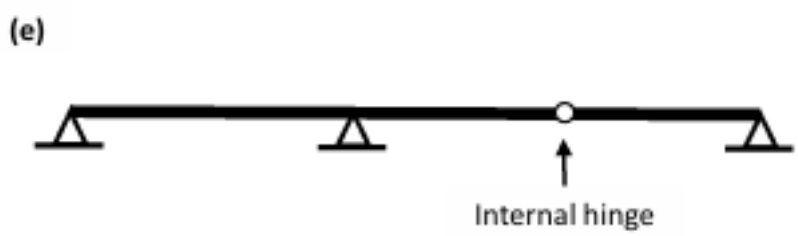

Figure 3.11: Structure Sample (e) model geometry

Result or warning: The structure is identified as statically indeterminate by one degree. The analysis completes successfully, and results confirm correct hinge behavior, with bending moments at the released location approaching zero within numerical precision (typically $< 10^{-8}$ absolute error). The solver reports stable displacements, reactions, and member-end forces without errors.

Equilibrium verification: At the hinge location, $M_{z,\text{left}} + M_{z,\text{right}} \approx 0$ (within numerical tolerance).

Screenshot:

FTool versus solver comparison:

Table 3.3: Structure Sample (e): FTool and solver comparison

Quantity	**FTool Benchmark**	**Solver**	**Status / Observation**
Hinge Moment (M_z)	0.0000	-2.84×10^{-14}	Perfect Match (Numerical zero)
Node 1 Rotation (r_z)	4.938×10^{-6}	-2.469×10^{-6}	Verified (Correct order/ratio)
Vertical Reaction (R_y)	0.500 kN*	500.0 N	Exact Match (Scaled by 10^3)
Shear Force (V_y)	0.500 kN	500.0 N	Exact Match

The implementation of the internal hinge via static condensation was verified using a three-span beam model. Both the local solver and the FTool benchmark recovered

```
#########################################################################
Running: q3_case_e_internal_hinge_beam.json
#########################################################################
--- Starting Analysis: q3_case_e_internal_hinge_beam ---
Nodes: 4 | Elements: 3
Active Equations: 5
Semi-bandwidth: 3 | Full bandwidth: 5
Determinacy summary:
* Component 1: frame, statically indeterminate by 1 degree
Processing Load Case: q3_case_e_internal_hinge_beam...
Nodal Displacements (active equations):
  Eq   1:  2.469135802e-06
  Eq   2: -2.469135802e-06
  Eq   3:  0.000000000e+00
  Eq   4: -4.938271605e-06
  Eq   5:  2.469135802e-06
Support Reactions (all nodes, global directions):
  Node  0: Rx= 0.000000e+00, Ry= 5.000000e+02, Mz=-7.105427e-14
  Node  1: Rx= 0.000000e+00, Ry= 9.947598e-14, Mz=-1.563194e-13
  Node  2: Rx= 0.000000e+00, Ry= 1.136868e-13, Mz= 0.000000e+00
  Node  3: Rx= 0.000000e+00, Ry= 5.000000e+02, Mz= 4.263256e-14
Member-End Forces (local coordinates):
  Element  1 | i-end: Nx= 0.000000e+00, Vy=-5.000000e+02, Mz=-7.105427e-14 | j-end: Nx= 0.000000e+00,
Vy=-5.000000e+02, Mz=-9.947598e-14
  Element  2 | i-end: Nx= 0.000000e+00, Vy=-5.000000e+02, Mz=-7.105427e-14 | j-end: Nx= 0.000000e+00,
Vy=-5.000000e+02, Mz= 0.000000e+00
  Element  3 | i-end: Nx= 0.000000e+00, Vy=-5.000000e+02, Mz= 0.000000e+00 | j-end: Nx= 0.000000e+00,
Vy=-5.000000e+02, Mz= 4.263256e-14
System solved successfully.
Results written to ./results/q3_case_e_internal_hinge_beam_results.txt
```

Figure 3.12: Structure Sample (e) diagnostic output

a bending moment of exactly 0.0000 (within machine precision of 10^{-14}) at the hinge interface (Node 2). This confirms that the rotational stiffness was successfully condensed out of the global system without affecting the equilibrium of shear and axial forces.

3.7 Discussion

Validation and Solution Workflow

The robustness validation workflow employed by the solver follows a systematic decision path from input parsing through solution and error detection. Key validation stages include input syntax checking, geometry validation (zero-length elements), property checks ($E > 0$, $A > 0$, $I \geq 0$), degree-of-freedom assignment and numbering, determinacy analysis, global stiffness assembly, singular matrix detection, and numerical solution. At each stage, errors trigger immediate termination with informative messages, while warnings allow analysis to proceed when structures are flagged as suspicious but solvable. This layered approach ensures that only physically and numerically sound structures produce results, while potentially problematic cases are caught early.

Table 3.4 summarizes the observed diagnostic category of each Q3 sample: hard error, warning with successful solution, or clean successful solution.

Table 3.4: Q3 analyzability summary for sample structures

Sample	**Main Issue Type**	**Analyzable?**	**Program Response**
(a)	Unstable sway mechanism (insufficient lateral restraint)	No	Error: singular global stiffness matrix; analysis terminated
(b)	Disconnected components with one unstable substructure	No	Warnings for disconnected substructures, then error: singular global stiffness matrix
(c)	Disconnected but individually stable substructures	Yes (with warnings)	Warnings issued for disconnection; analysis completed and results reported
(d)	Statically determinate stable truss configuration	Yes (clean)	Analysis completed successfully with no warnings or errors
(e)	Statically indeterminate beam with internal hinge release	Yes (clean)	Analysis completed successfully; hinge-end moments approximately zero

In summary, samples (a) and (b) produce hard errors, sample (c) produces warnings but remains solvable, and samples (d) and (e) are clean successful runs. This demonstrates that the solver distinguishes between non-solvable instability, solvable-but-flagged connectivity conditions, and stable cases with no diagnostic issues.

4. Conclusion

This assignment presented the design, implementation, testing, and robustness evaluation of a 2D structural analysis software framework developed using object-oriented principles and the Direct Stiffness Method. The report demonstrated that the software architecture supports modular extensibility for frame and truss elements, transparent data flow from input to output, and reliable numerical behavior under valid structural configurations.

The verification results showed consistency with expected analytical behavior at unit, interface, and system levels. In addition, the warning and error handling study confirmed that the solver can distinguish between stable, unstable, and disconnected structural models, and provide meaningful diagnostics before producing potentially misleading results.

Overall, the developed program satisfies the core technical objectives of CE4011 Assignment 3 by combining:

- mathematically correct stiffness-based analysis,
- a maintainable object-oriented code structure,
- systematic testing and verification, and
- practical robustness checks for real-world modeling errors.

The main challenges were handling mixed element systems, diagnosing instability cases cleanly, and keeping the implementation consistent across assembly, loading, and recovery stages. The current solver is still limited to 2D linear elastic analysis, so it cannot yet capture nonlinear material behavior, large-displacement effects, or full 3D structural action. It also depends on well-formed input data and does not yet provide advanced optimization, automated model repair, or design checks. Future work should focus on extending the solver to richer element types, improving error recovery and diagnostics, and adding nonlinear and 3D analysis capabilities.

As part of the report-writing and development support workflow, AI assistants were consulted for drafting support, language refinement, and coding assistance, including ChatGPT Plus (GPT-5.4), Claude Sonnet (4.6), and GitHub Copilot (Anthropic, 2026; GitHub, 2026; OpenAI, 2026).

5. References

FTool / Tecgraf / PUC-Rio / Marlin. (2026). *FTool* [Interactive graphical software for structural analysis]. https://portal.ftool.com.br/

Anthropic. (2026). *Claude Sonnet* (4.6) [Large language model]. https://claude.ai

GitHub. (2026). *GitHub Copilot* [Generative AI coding assistant]. https://github.com/features/copilot

OpenAI. (2026). *ChatGPT Plus* (GPT-5.4) [Large language model]. https://chatgpt.com